

基于 Tamarin 的 MQTT 协议安全性分析方法^{*}

郑红兵, 王焕伟, 赵 琪, 董姝岐, 井 靖[†]

(中国人民解放军战略支援部队信息工程大学 网络空间安全学院, 郑州 450001)

摘 要: MQTT 是物联网中被广泛应用的消息传输协议,其安全性问题备受关注。当前 MQTT 协议安全性分析主要面向协议实现平台,缺少面向协议标准的安全性测试,导致协议标准本身存在的安全缺陷难以发现。针对该问题,采用协议形式化分析技术,提出了一种基于 Tamarin 的 MQTT 协议安全性分析方法。该方法首先面向 MQTT 协议 3.1.1 标准,构建了协议状态机,并依据 Tamarin 语法规则,完成了形式化描述;然后针对保密属性和认证属性,给出了 MQTT 协议需要满足的安全属性引理描述;最后,基于 Dolev-Yao 威胁模型在 Tamarin 中完成了对 47 种协议安全属性的验证。结果显示有 9 种保密属性违反和 29 种认证属性违反,对结果进行攻击测试,验证了该方法对 MQTT 协议安全性分析的有效性,并提出了一种基于身份重认证的优化改进方案。

关键词: MQTT 协议; 保密属性; 认证属性; 形式化分析; Tamarin

中图分类号: TP309

文献标志码: A

文章编号: 1001-3695(2023)10-038-06

doi:10.19734/j.issn.1001-3695.2023.02.0038

Tamarin-based security analysis of MQTT protocol

Zheng Hongbing, Wang Huanwei, Zhao Qi, Dong Shuqi, Jing Jing[†]

(College of Cyberspace Security, PLA Strategic Support Force Information Engineering University, Zhengzhou 450001, China)

Abstract: The MQTT protocol is widely used in the Internet of Things for message transmission, and its security issues have received much attention. Currently, security analysis of the MQTT protocol mainly focuses on protocol implementation platforms, lacking security testing based on protocol standards, which makes it difficult to detect security vulnerabilities in the protocol standards themselves. To address this issue, this paper proposed a formal analysis method for MQTT protocol security based on Tamarin. The method firstly constructed a protocol state machine based on the MQTT protocol 3.1.1 standard, and completed formal description based on Tamarin syntax rules. Then, for confidentiality and authentication properties, the security properties that MQTT protocol needed to satisfy were given in the form of lemma descriptions. Finally, it completed the verification of 47 protocol security attributes based on the Dolev-Yao threat model in Tamarin. The results show that there are 9 confidentiality attribute violations and 29 authentication attribute violations. It conducted an attack test on the results, and the test results verify the effectiveness of the proposed method for MQTT protocol security analysis. It also proposed an optimized and improved scheme based on identity reauthentication.

Key words: MQTT protocol; confidentiality property; authentication property; formal analysis; Tamarin

0 引言

随着物联网、人工智能、6G 等新技术的飞速发展,构建“万物互联”的智慧地球将逐渐成为现实。根据 IoT Analytics 的调查报告,2022 年,全球活跃的物联网设备已达 144 亿台,预计 2025 年,将有 270 亿台物联网设备接入网络^[1],海量的设备接入和设备管理对网络带宽、通信协议以及平台服务架构带来了极大的挑战。MQTT 协议凭借其简单易实现,支持 QoS(服务质量)、报文小等特点,被广泛应用于智慧城市、智慧医疗、工业互联网等场景中。然而,根据 CVE 漏洞平台数据显示,目前已经发现多种针对 MQTT 协议的漏洞^[2],包括命令注入、文件上传、缓冲区溢出、堆溢出、未授权认证等。攻击者通过利用 MQTT 协议的安全漏洞发起攻击,可能会造成用户隐私泄露、设备被恶意控制、强制下线,严重的会造成设备崩溃。因此面向 MQTT 协议的安全性分析成为物联网安全领域当前研究的热点问题之一。

目前国内外研究人员对 MQTT 协议的安全性分析主要针

对协议实现的平台。Jia 等人^[3]通过对 MQTT 协议在物联网云平台进行研究,手动分析评估了平台在部署协议存在的漏洞。Ramos 等人^[4]在基于 MQTT 的安全解决方案中,提出了一种基于模板的模糊测试方法,对协议的安全性进行分析。文献[5]针对 MQTT 协议平台提出了一种基于攻击模式自动生成测试用例的方法,该方法将测试用例生成应用于 MQTT 协议实现的代理平台,手动分析了代理中的问题缺陷。Roets 等人^[6]对 MQTT 协议使用默认的配置,在 IoTPenn 程序上进行了安全性分析,发现了协议遭遇了拒绝服务、伪造身份非法访问等安全问题。文献[7,8]围绕 MQTT 协议在国内的部署情况展开研究,提出了一种黑盒模糊测试方法,针对 MQTT 协议客户端实现,验证了消息通过代理服务器转发实现攻击的可能性。Wang 等人^[9]对 MQTT、CoAP、AMQP 协议实现平台进行了安全性分析,开发了物联网应用层协议检测工具 MPIInspector,分析了不同平台协议实现层面的安全性。针对 MQTT 协议的安全性分析,仅仅面向协议实现层面分析,不能从根本上解决协议的安全隐患,因此需要对 MQTT 协议标准层面进行安全性验证。

收稿日期: 2023-02-03; **修回日期:** 2023-04-04 **基金项目:** 国家重点研发项目(2019QY502)

作者简介: 郑红兵(1995-),男,河南平顶山人,硕士研究生,主要研究方向为物联网安全;王焕伟(1987-),男,河南周口人,讲师,博士研究生,主要研究方向为物联网安全、协议安全;赵琪(1999-),男,河南三门峡人,主要研究方向为物联网安全;董姝岐(1997-),女,河南商丘人,硕士研究生,主要研究方向为物联网安全;井靖(1980-),女(通信作者),河南郑州人,副教授,硕导,博士,主要研究方向为物联网安全、信息物理系统安全(jingjing_cs@hotmail.com)。

针对 MQTT 协议缺乏面向协议标准的安全分析问题,本文提出了一种基于 Tamarin^[10] 的 MQTT 协议安全性分析方法。该方法基于 MQTT 3.1.1 标准^[11],用符号化的语言对协议状态机^[12]、安全目标属性以及攻击者模型进行形式化建模^[13]。利用模型检测工具 Tamarin 对协议目标属性进行了安全性验证和结果分析,并在具体协议部署的环境中对分析结果进行了验证。最后,提出了协议优化改进方案。

1 MQTT 协议概述

消息队列遥测传输(message queuing telemetry transport, MQTT)协议是一种轻量级的消息传递协议。本文分析及验证的对象采用当前服务支持和使用最广泛的3.1.1版本。

1.1 MQTT 协议对象模型

MQTT 协议采用发布订阅式的体系结构,由订阅者(subscriber)、发布者(publisher)以及代理(broker)三个主要实体组成,如图 1 所示。在这种架构下,温湿度等传感器作为发布者,通过 MQTT 协议将感知的数据传递给 MQTT 代理,代理再将信息发送给订阅者,订阅者可以是移动终端、可编程逻辑控制器 PLC 等。需要注意的是,不同的应用场景下,客户端设备既可以是发布者,也可以是订阅者。这种体系结构的优点在于,发布者和订阅者之间的通信是松耦合的,发布者不需要知道订阅者的身份,也不需要知道订阅者何时接收信息,而订阅者也不需要知道发布者的身份。代理作为中间人,负责管理发布者和订阅者之间的通信,实现了高效的信息传递和管理。同时, MQTT 协议还支持 QoS,可以保证信息的可靠传递。

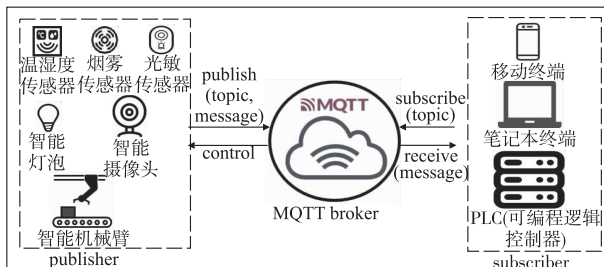


图 1 MQTT 协议交互模型

Fig. 1 MQTT protocol communication model

1.2 MQTT 协议交互过程

MQTT 协议规定通信双方按照发布者和订阅者的角色完成数据交互,交互过程包括设备注册、密钥协商、连接、订阅/取消订阅、发布和断开连接六个部分。

a) 设备注册。发布者和订阅者在信息传递之前在服务器中进行注册,服务器为发布、订阅设备分配唯一的客户端标识符(clientID)。

b) 密钥协商。发布和订阅设备通过密钥协商确定客户端设备的用户名和密码以及临时密钥 key, 用于对客户端设备进行身份验证和权限授权。

c) 设备与服务器连接包括设备连接和连接确认。设备向代理发送 connect 请求,进行设备连接。如果 connect 消息格式不正确或发送连接超时,连接关闭,防止消耗代理服务器资源。代理服务器收到 connect 请求后,对发送连接请求的设备进行认证和授权,并发送 ConnAck 响应消息。

d) 订阅和取消订阅过程。订阅客户端和代理服务器建立连接后,向代理服务器发送 Subscribe 消息进行主题订阅。为了确定每一个订阅,代理服务器向客户端发送 SubAck 确认消息数据包。与订阅消息相对应的是 Unsubscribe 消息,代理服务器接收到取消订阅后删除设备的订阅,同时代理向订阅设备发送一个 UnsubAck 确认消息。

e) 信息发布过程。如图 2 所示,发布设备在收到 ConnAck 消息后,发送 publish 数据包,包含主题(topic)、有效载荷、QoS,以及 deviceID 以字节格式传输的数据。为了保证消息的可靠传输,MQTT 协议提供了三种 QoS 供用户选择。QoS 0 依赖底层的 TCP/IP 网络来传递消息;QoS 1 服务器的消息接收由 PUBACK 消息进行确认;QoS 2 确保消息到达一次,并且消息不会重复发送,订阅者需要向发布者发送 Pubrec 数据包,代理转发后,发布者发送 Pubrel 数据包,订阅者接收后发送 Pubcomp 来结束信息发布,保证实时性交互。具体信息发布过程如图 2 所示。

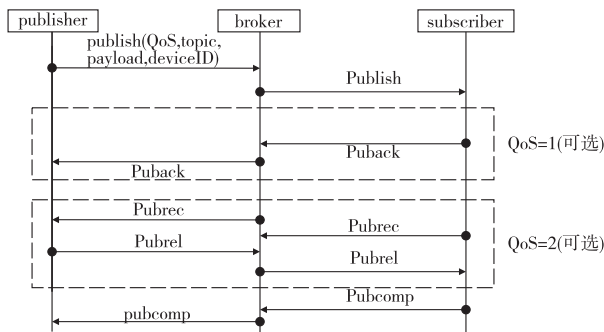


图2 消息发布过程

Fig. 2 Message distribution process

f)设备断开连接。发布和订阅设备向代理服务器发送 disconnect 消息,断开与服务器的连接。

2 MQTT 协议形式化建模

本文采用协议形式化建模分析工具 Tamarin 对 MQTT 协议进行形式化建模,使用安全协议理论语言 (sphy) 描述协议状态转移的过程,使用 rule 来表示协议的状态,使用 lemma 来描述需要验证的安全属性。下面分别描述对协议状态机、安全属性、攻击者模型的建模过程。

2.1 MQTT 协议状态机建模

协议状态机是对协议流程的动态化描述,是用来进行协议行为建模的有力工具,它描述了协议实体在它的生命周期内所经历的状态序列,以及如何响应来自外界的各种事件。

在协议的模型中包括代理服务器、发布设备和订阅设备三类实体。根据 MQTT 协议标准对代理服务器、发布设备和订阅设备进行分析描述,构建了协议运行过程的状态变化。服务器状态机交互如图 3 所示。

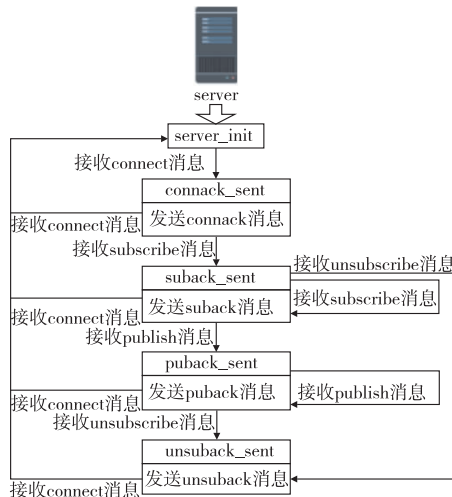


图3 MOTT 协议服务器状态机

Fig. 3 MQTT protocol server state machine

服务器在接收到连接请求后,会将其从初始状态(server

init) 转换为连接确认发送状态 (connack_sent), 随后在接收到设备订阅消息时, 会将其转换为订阅确认状态 (suback_sent)。此外, 服务器还能够接收其他设备发送的订阅 (publish)、连接 (connect)、取消订阅 (unsuback) 等消息, 并在接收到相关消息后转换为相应的确认发送状态。因此, 服务器构成的状态集合为 $\{ \text{server_init}, \text{connack_sent}, \text{suback_sent}, \text{puback_sent}, \text{unsuback_sent} \}$ 。

发布设备交互过程如图 4 所示, 包括了连接、发布、取消发布以及断开连接的状态交互变化。

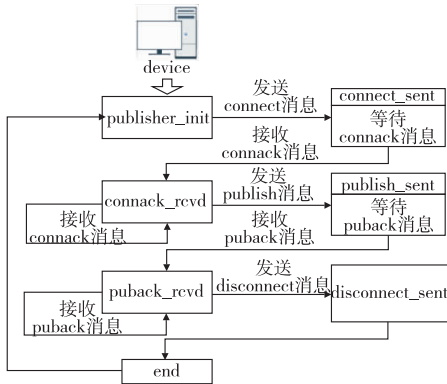


图 4 MQTT 协议发布设备状态机

Fig. 4 MQTT protocol publishing device state machine

发布设备初始状态 (publisher_init) 在发送连接消息后转换为连接发送状态 (connect_sent) 并等待接收连接确认消息 (connack_sent), 转换为接收连接确认状态 (connack_rcvd)。在发布 (publish) 消息后, 客户端状态依次转换为订阅发送状态 (publish_sent), 发布确认接收状态 (puback_rcvd) 以及断开连接发送状态 (disconnect_sent) 如图 4 所示, 发布设备构成的状态集合为 $\{ \text{publisher_init}, \text{connect_sent}, \text{connack_rcvd}, \text{publish_sent}, \text{puback_rcvd}, \text{disconnect_sent} \}$ 。

订阅设备交互过程如图 5 所示, 包括连接、订阅、取消订阅、断开连接的状态交互变化。

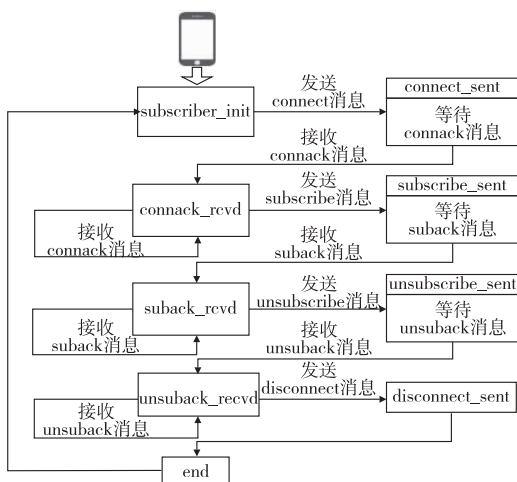


图 5 MQTT 协议订阅设备状态机

Fig. 5 MQTT protocol subscribing device state machine

订阅设备初始状态 (subscriber_init) 发送连接消息后转换为连接发送状态 (connect_sent) 并等待接收连接确认消息 (connack_sent), 转换为接收连接确认状态 (connack_rcvd)。在发送订阅 (subscribe)、取消订阅 (unsubscribe) 消息后, 客户端状态依次转换为订阅发送状态 (subscribe_sent)、取消订阅状态 (unsubscribe_sent), 如图 5 所示, 客户端设备构成的状态集合为 $\{ \text{subscriber_init}, \text{connect_sent}, \text{connack_sent}, \text{connack_rcvd}, \text{subscribe_sent}, \text{suback_sent}, \text{suback_rcvd}, \text{unsubscribe_sent}, \text{unsuback_rcvd}, \text{disconnect_sent} \}$ 。

服务器和客户端设备状态机初始化在 Tamarin 中描述如下, 其中 rule 表示对协议状态的描述:

```
rule init_server:
  "[ ] -- [ Start_Server_1 ( $Server ) ] -> [ St_sev_prev_0_Init ( $Server ), ! Server ( $Server ) ]"

rule init_dev:
  "let
    pkDev = pk ( ~ skDev )
    credentials = < ~ key, ~ skDev, pkDev >
    clientid = < $registryid, ~ deviceid >
    context = $Server
  in
    [ Fr ( ~ skDev ), Fr ( ~ deviceid ), Fr ( ~ key ), ! Server ( $Server ) ] --
    [ Start_Dev_1 ( ~ deviceid ), Src_NotKey ( ~ skDev, ~ deviceid, ~ key ) ] ->
    [ ! St_dev_0_Init ( clientid, credentials, context ), Factory_Provision ( clientid, pkDev ) ]"
```

2.2 安全属性建模

本文从协议标准规范出发, 归纳了参与协议交互实体需要满足的保密属性和认证属性。在认证属性方面, 考虑到参与协议交互双方的身份认证需求, 本文采用文献 [14] 的定义进行分类, 主要包括存活性 (aliveness)、弱一致性 (weak agreement)、非单射一致性 (noninjective agreement) 和单射一致性 (injective agreement)。在保密属性方面, 考虑到协议实体交互过程中信息的私密性, 主要涉及设备密钥以及主题 (topic) 和负载 (payload) 的保密性。其中, 主题保密性包括发布主题、订阅主题以及取消订阅主题的保密性。

2.2.1 认证属性

根据协议规范, 交互实体分为发布设备、订阅设备和代理服务器三类。根据这三类实体发送的不同数据包类型, 将操作分为连接、取消连接、订阅、取消订阅和发布五种。此外, 代理服务器向客户端设备发送的操作也可以分为连接确认、订阅确认、取消订阅确认和发布确认四种。基于这些操作, 本文可以对 36 种认证属性进行形式化建模, 包括存活性、弱一致性、非单射一致性和单射一致性。认证属性如表 1 和 2 所示。

为了说明建模过程, 下面以订阅设备向代理服务器发送数据为例, 详细介绍 MQTT 协议交互过程中连接 devid 单射一致性、订阅的存活性、取消订阅的弱一致性以及断开连接的非单射一致性的认证属性建模过程, 安全属性在 Tamarin 中用 lemma 表示。

表 1 设备到服务器的认证属性
Tab. 1 Authentication properties from device to server

对象	操作	属性
Pub, Sub- > Serv	connect	aliveness
		weak agreement
		noninjective agreement
	disconnect	injective agreement
		aliveness
		weak agreement
Sub- > Serv	subscribe	noninjective agreement
		injective agreement
		aliveness
	unsubscribe	weak agreement
		noninjective agreement
		injective agreement
Pub- > Serv	puback	aliveness
		weak agreement
		noninjective agreement

表 2 服务器到发布设备的认证属性
Tab. 2 Authentication properties from server to device

对象	操作	属性
Serv- > Dev	connack	aliveness
		weak agreement
		noninjective agreement
		injective agreement
	suback	aliveness
		weak agreement
		noninjective agreement
		injective agreement
	unsuback	aliveness
		weak agreement
		noninjective agreement
		injective agreement
	puback	weak agreement
		noninjective agreement
		injective agreement

1) 订阅设备到服务器 devid_connect 的单射一致性

代理服务器和发布客户端在进行 connect 的过程中 devid 的单射一致性。代理服务器认为自己与客户端设备完成了一次协议数据交换,并且数据交换是一一对应的,协议参与的实体是代理服务器和客户端设备,攻击者没有获取数据交换的密钥,且只存在一次协议的运行,代理服务器认为与设备进行的通信消息没有被重放。在 Tamarin 中形式化建模为

```
lemma injective_agreement_serv_dev_devid_CONNECT;
```

```
" All a b t #i.
```

```
Commit(a,b,<'serv','dev','devid',t>)) @ i == > (Ex #j. Running(b,a,<'serv','dev','devid',t>)) @ j & j < i & not (Ex a2 b2 #i2. Commit(a2,b2,<'serv','dev','devid',t>)) @ i2 & not (#i2 = #i)) | (Ex C data #r. Reveal(C,data) @ r & Honest(C) @ i)"
```

2) 订阅设备向服务器订阅的存活性

客户端设备向代理服务器发送订阅请求,代理服务器对请求回应。存活性要求除了在此时间点以外,设备和服务器都可以完成过 MQTT 协议运行,防止设备掉线。在 Tamarin 中形式化建模为

```
lemma aliveness_serv_dev_SUBSCRIBE;
```

```
" All a b t #i.
```

```
Commit(a,b,<'serv','dev','subTopic',t>)) @ i == > (Ex id #j. Create('sub','dev',id) @ j) | (Ex C data #r. Reveal(C,data) @ r & Honest(C) @ i)"
```

3) 订阅设备向服务器取消订阅的弱一致性

设备向代理服务器发送取消订阅主题的消息完成一次信息交互,代理服务器认为设备的身份可信。在 Tamarin 中形式化建模为

```
lemma weakagreement_serv_dev_UNSUBSCRIBE;
```

```
" All a b t #i.
```

```
Commit(a,b,<'serv','dev','unsubTopic',t1>)) @ i == > (Ex t2 #j. Running(b,a,<'serv','dev','unsubTopic',t2>)) @ j) | (Ex C data #r. Reveal(C,data) @ r & Honest(C) @ i)"
```

4) 订阅设备与服务器断开连接的非单射一致性

代理服务器认为自己与设备端完成了一次协议的运行,至少存在一次协议运行,代理服务器认为协议之间传输的消息是真实设备提供的,消息没有被篡改。在 Tamarin 中形式化建模为

```
lemma noninjective_agreement_serv_dev_DISCONNECT;
```

```
" All a b t #i.
```

```
Commit(a,b,<'serv','dev','discon',t>)) @ i == > (Ex #j. Running(b,a,<'serv','dev','discon',t>)) @ j) | (Ex C data #r. Reveal(C,data) @ r & Honest(C) @ i)"
```

2.2.2 保密属性

保密属性分别从发布设备、订阅设备和代理服务器的角度对设备密钥(secret_key)、客户端编号(clientid)、发布主题及发布负载、订阅主题和取消订阅主题内容等方面对期望协议满足的保密属性进行建模,如表 3 所示。

表 3 保密属性
Tab. 3 Confidentiality properties

对象	属性
Dev	secret_Key
	clientID
	publishpayload
	publishTopic
	subscribeTopic
Serv	unsubscribeTopic
	clientID
	publishpayload
	publishTopic
	subscribeTopic
	unsubscribeTopic

下面以客户端设备为例介绍保密属性的建模过程。

1) 设备私钥 sk 的保密属性

从客户端的角度考虑私钥 sk 的安全性,若客户端声明会话密钥是保密的,并且攻击者无法获取其私钥 sk,则攻击者不能够造成加密信息泄露,只有合法的目标可以实现消息的解密,在 Tamarin 中形式化建模为

```
lemma secrecy_dev_Secret_Key;
```

```
" All dev msg #i. Secret(<'dev',dev>,'sk',msg) @ i == > (not (Ex #j. K(msg) @ j)) | (Ex X data #j. Reveal(X,data) @ j & Honest(X) @ i)"
```

2) 设备发布主题的保密属性

如果攻击者获取了设备需要发布的主题,能够对相关主题的服务器实施拒绝服务攻击,导致代理服务器无法接收正常设备的发布请求,因此发布设备在向代理服务器进行通信的时候必须要保证自己的发布主题是秘密的。在 Tamarin 中形式化建模为

```
lemma secrecy_dev_PublishTopic;
```

```
" All dev msg #i. Secret(<'dev',dev>,'pubTopic',msg) @ i == > (not (Ex #j. K(msg) @ j)) | (Ex X data #j. Reveal(X,data) @ j & Honest(X) @ i)"
```

2.3 攻击者建模

在协议形式化安全性分析中,有 eCK^[15] 和 Dolev-Yao^[16] 两种常用的攻击者模型。其中,eCK 模型是安全性较强的形式化模型,攻击者可以获取协议参与方的临时密钥,因此在该模型下证明安全的协议具有极高的安全保证。但是,eCK 模型进行安全证明的过程需要使用双线性映射,导致计算效率较低。相比之下,Dolev-Yao 模型能够更好地模拟 MQTT 环境中的威胁,且代码量较少,进行协议分析时效率更高。因此,本文选择 Dolev-Yao 攻击者模型对攻击者进行建模。

在 Dolev-Yao 攻击者模型中,攻击者的能力主要包括以下几个方面:a)可以在不被察觉的情况下,嗅探公开信道中的协议通信数据;b)攻击者可以丢弃、重构公共信道中传输的消息,从而破坏通信的完整性;c)攻击者可以假冒合法的参与主体,参与协议通信,从而破坏通信的真实性;d)攻击者遵循所有的密码学加密规范,只有在掌握密钥的情况下才可以对消息进行解密。

上述攻击者能力在 Tamarin 中形式化建模为

```
" [Out(x)] -> [K(x)], [Fr(x)] -> [K(x)]; [! KU(x)] -> [In(x)]; [K(x)] -> [In(x)]"
```

3 验证及结果分析

基于前文构建的 MQTT 协议状态机、安全属性以及攻击者模型,利用 Tamarin 可以完成协议安全属性的验证。Tamarin 的验证过程如图 6 所示,Tamarin 的输入为 rule 和 lemma,通过模拟协议通信交互过程,将所有交互的情况进行遍历,结合定义的期望满足的安全属性进行检测,验证是否存在属性违反,若验证通过(true),表示满足安全属性,否则验证不通过(false),输出反例及攻击路径。验证平台采用 16 核 CPU (Intel® Core™ i7-

10875H 2.30 GHz), 16 GB 内存, 操作系统为 Ubuntu 20.04, Tamarin 版本为 1.6.1。

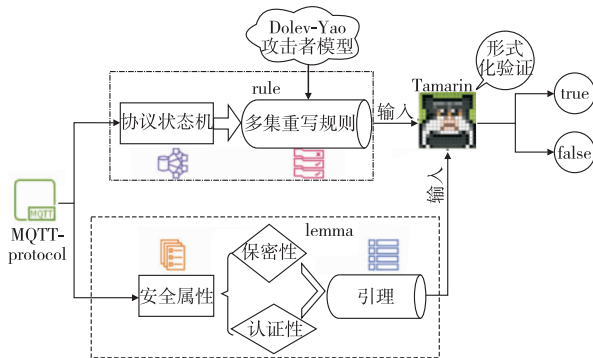


图 6 Tamarin 证明器工作原理
Fig. 6 Tamarin prover working principle

3.1 保密属性验证结果及分析

通过将协议状态机模型及 11 条保密属性的形式化描述输入 Tamarin 进行验证, 验证过程如图 7 所示, 其中左边绿色部分表示已经验证通过的 lemma, 红色的箭头表示进行攻击的过程 (见电子版)。

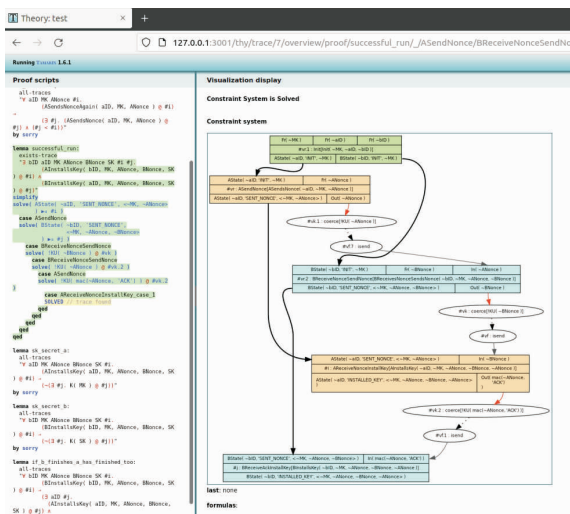


图 7 Tamarin 验证过程
Fig. 7 Tamarin verification process

表 4 保密属性验证结果

Tab. 4 Confidentiality property verification results

类别	Dev	Serv	类别	Dev	Serv
secret_Key	✓	/	publishTopic	✓	×
clientID	×	×	subscribeTopic	×	×
publishpayload	×	×	unsubscribeTopic	×	×

表 4 展示了协议参与实体发布设备、订阅设备和代理服务器在 secret_key、clientID、publishpayload、publishTopic、subscribeTopic、unsubscribeTopic 上的 11 种保密属性的验证情况, 其中“✓”表示满足, “×”表示不满足, “/”表示不涉及。由结果可知, 11 种保密属性中有 2 种满足, 9 种不满足。

设备的 secret_key 和 publishTopic 的保密性没有违反, 原因在于设备密钥和发布主题具有较强的可以选择性与私密性, 攻击者无法提前窃取相关的隐私信息, 而 clientID、publishpayload、subscribeTopic、unsubscribeTopic 分别遭受了属性违反, 易遭到中间人攻击和拒绝服务攻击等。

a) clientID 属性违反, 攻击者冒充受害者设备连接到代理服务器, 发送非法消息数据包进行破坏, 特别是当服务器端检测到相同的 clientID 设备, 服务器会断开与当前客户端的连接, 迫使受害者下线。

b) subscribeTopic 属性违反, 攻击者可以利用自己的 clientID 进行主题订阅, 非法获取订阅的相关信息, 当受害者设备发送其秘密消息时, 代理服务器将数据传输给攻击者。

c) publishPayload 属性违反, 攻击者也可以通过拦截设备发布的信息, 对 publishPayload 内容进行修改, 植入木马文件, 通过代理服务器的转发将恶意的载荷发送给订阅设备进行攻击。

d) unsubscribeTopic 属性违反, 攻击者能够将取消订阅消息发送给服务器, 迫使受害者无法恢复对相关主题的订阅, 影响设备的正常工作。

3.2 认证属性验证结果及分析

通过将协议状态机模型及 36 条认证属性的形式化描述输入 Tamarin 进行验证, 验证结果如表 5 所示。

表 5 展示了 MQTT 协议参与实体的 connect、disconnect、subscribe、publish、connack、suback、unsuback、puback 操作的存活性、弱一致性、非单射一致性、单射一致性认证属性的验证结果, 共验证了 36 种认证属性, 其中有 29 种属性遭到违反, 7 种安全属性得到满足。

表 5 认证属性验证结果

Tab. 5 Authentication attribute verification results

类别	Dev -> Serv					Serv -> Dev			
	connect	disconnect	subscribe	unsubscribe	publish	connack	suback	unsuback	puback
存活性	✓	×	×	✓	×	×	×	×	×
弱一致性	✓	×	×	✓	×	×	×	×	×
非单射一致性	✓	×	×	✓	×	×	×	×	×
单射一致性	×	×	×	✓	×	×	×	×	×

从实验结果中分析可知, 从设备端到代理服务器端, 除去 unsubscribe 的操作, 其他的协议运行操作都存在不同程度的认证属性违反, 存在一定的安全风险。下面重点分析从设备端到服务器的通信交互过程中属性违反安全风险等级最高的三种情况:

a) connect 操作单射一致性没有得到保证, 说明攻击者可以收集客户端向代理服务器发送的历史信息, 并重新发送到服务器上, 进行重放攻击。

b) subscribe 操作、publish 操作认证属性全部违反, 攻击者可以对设备发送的信息进行篡改, 劫持客户端身份, 发送精心构建的消息负载, 如木马文件, 将代理服务器作为“跳板”转发给订阅主题的设备, 实施中间人攻击。

c) disconnect 操作认证属性违反, 攻击者可以重复发送断

开连接消息, 导致设备无法进行重新连接, 实施拒绝服务攻击。

从代理服务器到客户端的角度, connack、suback、unsuback、puback 的认证属性全部违反, 反映出在公共信道上的通信更容易遭受攻击。攻击者可以在服务器和客户端之前的信道上随意发送伪造代理的数据包, 而订阅设备和发布设备由于资源受限并不能分辨代理服务器真伪, 造成设备被非法控制, 导致服务器到客户端之间的认证属性遭到破坏。

3.3 分析结果验证

针对发现的 MQTT 协议安全隐患, 通过配置具体的协议应用场景进行安全性验证。验证实验环境采用 16 核 CPU (Intel® Core™ i7-10875H 2.30 GHz), 16 GB 内存, 操作系统为 Ubuntu 20.04, 选择 MQTT 协议开源软件 Mosquitto v2.0.15 作为 MQTT 协议代理服务器, 选择 MQTTX v1.9.1 作为协议信

息交互的订阅者和发布者,选择渗透测试工具 Burp suite^[17]作为攻击者实施攻击测试(包括数据截获、数据篡改、数据利用、

身份劫持等),测试结果如表 6 所示。其中:“/”表示不涉及;“√”表示存在。

表 6 安全分析结果验证
Tab. 6 Verification of security analysis results

类别	clientID	subscribeTopic	publishPayload	unsubscribeTopic	connect	publish	subscribe	disconnect	属性违反
数据截获	√	√	√	√	/	/	/	/	保密属性
数据篡改	/	/	√	/	/	/	/	/	保密属性
数据利用	√	/	/	/	√	√	√	√	认证属性
身份劫持	√	/	/	/	/	/	/	/	认证属性

根据测试结果,部署 MQTT 协议 3.1.1 标准的平台,利用渗透测试工具可以实施数据截获、篡改、利用及身份劫持。其中数据截获、篡改、违反了协议的保密属性,数据利用和身份劫持违反了协议的认证属性。测试结果验证了本文基于 Tamarin 的 MQTT 协议安全性分析方法的有效性。

4 基于身份重认证的优化方案

现有 MQTT 协议安全加固的方法是基于特定的实现框架(如 OAuth 1.0a)^[18]和基于 AugPAKE 协议的 MQTT 安全解决方案^[19],适用场景有限,无法从根本上解决 MQTT 协议的安全隐患。本文基于 Tamarin 对 MQTT 协议的形式化验证结果,提出了一种基于身份重认证的优化方案,该方案主要包括密钥重计算和随机会话标识符生成两部分,重点用来解决通信交互双方身份确认问题和会话竞争问题。

1) 密钥重计算

首先,客户端设备将通信双方在密钥协商阶段已经产生的共享密钥 key 与连接消息组合;然后,通过消息验证码算法,将加密后的消息发送给代理服务器,服务器通过计算 MAC 确定设备身份,将 connack 数据包与 key 重新计算 MAC 发送给客户端设备,实现客户端设备与代理服务器的身份认证。这里选择基于 hash 的 MD5 的 MAC 加密算法^[20]对消息进行加密,也称为 HMAC-MD5,加密计算公式如下:

$$\text{HMAC}(K, M) = H((K \text{ XOR } \text{opad}) + H((K \text{ XOR } \text{ipad}) + M))$$

其中:H 是 hash 函数 MD5;K 是密钥;M 是认证的消息;+ 代表“连接”运算;XOR 代表异或运算;opad 是外部的填充常数;ipad 是内部填充常数。添加的改进部分描述为

Dev→Broker: < ConnectDev > = < "connect", HMAC (~key, "Connect") >

Broker→Dev: < ConAckBroker > = < "connack", HMAC (~key, "Connack") >

改进后的协议交互流程如图 8 所示。

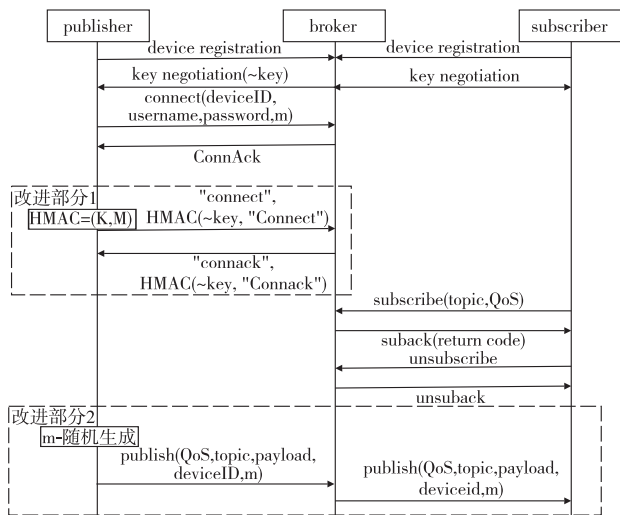


图 8 改进方案

Fig. 8 Improvement program

2) 随机会话标识符生成

针对发布设备和代理服务器缺少会话绑定而导致的会话

竞争问题,可能会导致客户端设备发送的信息无法被代理服务器接收,从而无法满足弱一致性的要求。为了解决这个问题,可以在客户端和代理之间的 publish 会话中添加随机生成的会话标识符 m。通过消息中的会话标识符,可以识别来自不同客户端设备的两个相同 deviceID 发送的消息,从而避免相同 deviceID 之间的资源争夺,进而减少攻击者伪造合法用户身份进行信息窃取的可能性。

上述的协议改进方案能够解决协议交互过程中双方无法确认身份的问题,并且能够减少攻击者窃取用户身份争夺服务器资源造成正常用户无法正常使用的问题。此外,改进方案的两部分可根据需要灵活选择,若设备资源有限,只需在通信交互过程中添加随机生成会话标识符,若设备资源充足,可选择密钥重计算和随机标识符生成同时部署。

5 结束语

本文针对 MQTT 3.1.1 协议标准,对 MQTT 协议状态机和期望满足的安全属性进行了形式化建模,在 Tamarin 中验证了 MQTT 协议 11 种保密属性和 36 种认证属性的满足情况,发现了 38 种属性违反,评估了 MQTT 协议在标准层面可能面临的安全风险并进行验证。针对面临的安全属性违反的情况,提出了一种基于身份重认证的认证属性优化方案。

下一步工作,一方面,利用 Tamarin 完成改进后的 MQTT 协议的有效性验证,并从时间消耗和资源消耗等方面,详细分析改进方案的效率问题;另一方面,把改进方案与其他改进方案进行详细比较分析,完成改进方案的优化工作。此外,为了进一步提高形式化验证方法在协议安全性分析中的效率,基于机器学习的协议自动化建模方法也是未来值得研究的内容。

参考文献:

- [1] Hasan M. State of IoT 2022: number of connected IoT devices growing 18% to 14.4 billion globally [EB/OL]. (2022-05-19). <https://iot-analytics.com/number-connected-iot-devices/>.
- [2] U. S. Department of Homeland Security (DHS) Cybersecurity and Infrastructure Security Agency (CISA). CVE records [EB/OL]. [2022-10-18]. <https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=mqtt>.
- [3] Jia Yan, Xing Luyi, Mao Yuhang, et al. Burglars' IoT paradise: understanding and mitigating security risks of general messaging protocols on IoT clouds [C]//Proc of IEEE Symposium on Security and Privacy. Piscataway, NJ: IEEE Press, 2020: 465-481.
- [4] Ramos S H, Villalba M T, Lacuesta R. MQTT security: a novel fuzzing approach [J]. Wireless Communications and Mobile Computing, 2018, 2018: article ID 8261746.
- [5] Sochor H, Ferrarotti F, Ramler R. Automated security test generation for MQTT using attack patterns [C]//Proc of the 15th International Conference on Availability, Reliability and Security. New York: ACM Press, 2020: article No. 97.
- [6] Roets A, Tait B L. IoT-Penn: a security penetration tester for MQTT in the IoT environment [M]//Jahankhani H. Cybersecurity in the Age of Smart Societies. Cham: Springer, 2023: 141-157.
- [7] 徐绘凯, 刘跃, 马振邦, 等. MQTT 安全大规模测量研究 [J]. 信息网络安全, 2020, 20(9): 37-41. (Xu Huikai, Liu Yue, Ma Zhenbang, et al. MQTT security large-scale measurement research [J]. Information Network Security, 2020, 20(9): 37-41.)

(下转第 3143 页)

- tunnel detection approach[J]. *Procedia Computer Science*, 2013, 17:852-860.
- [4] Farnham G, Atlasis A. Detecting DNS tunneling[J]. *SANS Institute InfoSec Reading Room*, 2013, 9:1-32.
- [5] 刁嘉文, 方滨兴, 田志宏, 等. 基于攻击流量自生成的DNS隐蔽信道检测方法[J]. *计算机学报*, 2022, 45(10):2190-2206. (Diao Jiawen, Fang Binxing, Tian Zhihong, et al. DNS covert channel detection method based on attack traffic self generation[J]. *Chinese Journal of Computers*, 2022, 45(10):2190-2206.)
- [6] Das A, Shen Minyi, Shashanka M, et al. Detection of exfiltration and tunneling over DNS[C]//Proc of the 16th IEEE International Conference on Machine Learning and Applications. Piscataway, NJ: IEEE Press, 2017:737-742.
- [7] Born K, Gustafson D. Detecting DNS tunnels using character frequency analysis[EB/OL]. (2010). <https://arxiv.org/abs/1004.4358>.
- [8] Preston R. DNS tunneling detection with supervised learning[C]//Proc of IEEE International Symposium on Technologies for Homeland Security. Piscataway, NJ: IEEE Press, 2019:1-6.
- [9] Wu Kemeng, Zhang Yongzheng, Yin Tao. CLR: a classification of DNS tunnel based on logistic regression[C]//Proc of the 38th IEEE International Performance Computing and Communications Conference. Piscataway, NJ: IEEE Press, 2019.
- [10] Chen Yang, Li Xiaoyang. A high accuracy DNS tunnel detection method without feature engineering[C]//Proc of the 16th International Conference on Computational Intelligence and Security. Piscataway, NJ: IEEE Press, 2020:374-377.
- [11] Ellens W, Żurawski P, Sperotto A, et al. Flow-based detection of DNS tunnels[C]//Proc of IFIP International Conference on Autonomous Infrastructure, Management and Security. Berlin: Springer, 2013: 124-135.
- [12] D'Angelo G, Castiglione A, Palmieri F. DNS tunnels detection via DNS-images[J]. *Information Processing & Management*, 2022, 59(3):102930.
- [13] Bai Huiwen, Liu Weiwei, Liu Guangjie, et al. Application behavior identification in DNS tunnels based on spatial-temporal information[J]. *IEEE Access*, 2021, 9:80639-80653.
- [14] Zhan Mengqi, Li Yang, Yu Guangxi, et al. Detecting DNS over HTTPS based data exfiltration[J]. *Computer Networks*, 2022, 209: 108919.
- [15] Jiang Kui, Wang Fei. Detecting DNS tunnel based on multidimensional analysis[C]//Proc of the 5th International Conference on Mechanical, Control and Computer Engineering. Piscataway, NJ: IEEE Press, 2020:272-275.
- [16] Bai Huiwen, Liu Guangjie, Zhai Jiangtao, et al. Refined identification of hybrid traffic in DNS tunnels based on regression analysis[J]. *ETRI Journal*, 2021, 43(1):40-52.
- [17] Nadler A, Aminov A, Shabtai A. Detection of malicious and low throughput data exfiltration over the DNS protocol[J]. *Computers & Security*, 2019, 80:36-53.
- [18] Liu Yong, Gou Xiangnan. Research on application of feature analysis method in DNS tunnel detection[C]//Proc of the 5th Annual International Conference on Information System and Artificial Intelligence. [S. l.]: IOP Publishing, 2020:012069.
- [19] 沈传鑫, 王永杰, 熊鑫立. 基于图注意力网络的DNS隐蔽信道检测[J]. *信息安全*, 2023, 23(1):73-83. (Shen Chuanxin, Wang Yongjie, Xiong Xinli. DNS covert channel detection based on graph attention networks[J]. *Netinfo Security*, 2023, 23(1):73-83.)
- [20] Ishikura N, Kondo D, Vassiliades V, et al. DNS tunneling detection by cache-property-aware features[J]. *IEEE Trans on Network and Service Management*, 2021, 18(2):1203-1217.
- [21] Chen Shaojie, Lang Bo, Liu Hongyu, et al. DNS covert channel detection method using the LSTM model[J]. *Computers & Security*, 2021, 104:102095.
- [22] Wang Yue, Zhou Anmin, Liao Shan, et al. A comprehensive survey on DNS tunnel detection[J]. *Computer Networks*, 2021, 197:108322.
- [23] Data exfiltration toolkit[EB/OL]. (2016-06-13) [2017-11-03]. <https://github.com/sensepost/DET>.
- [24] Dnscat2[EB/OL]. (2014) [2018-06-26]. <https://github.com/iagox86/dnscat2>.
- [25] Iodine[EB/OL]. (2010) [2014-06-16]. <https://code.kryo.se/iodine/>.
- [26] dns2tcp[EB/OL]. (2016) [2020-03-07]. <https://github.com/zf19/dns2tcp>.
- [27] TCP-over-DNS[EB/OL]. (2008) [2008-07-27]. <https://analogbit.com/software/tcp-over-dns>.
- [28] Reverse_DNS_Shell[EB/OL]. (2014) [2015-12-11]. https://github.com/ahhh/Reverse_DNS_Shell.
- [29] DNSExfiltrator[EB/OL]. (2017) [2018-04-06]. <https://github.com/Arno0x/DNSExfiltrator>.
- [30] DNSlivery[EB/OL]. (2018) [2019-02-01]. <https://github.com/no0be/DNSlivery>.
- [31] goDoH[EB/OL]. (2018) [2022-06-26]. <https://github.com/sensepost/godoh>.
- (上接第3137页)
- [8] Xu Huikai, Yu Miao, Wang Yanhao, et al. Trampoline over the air: breaking in IoT devices through MQTT brokers[C]//Proc of the 7th European Symposium on Security and Privacy. Piscataway, NJ: IEEE Press, 2022:171-187.
- [9] Wang Qinying, Ji Shouling, Tian Yuan, et al. MPIInspector: a systematic and automatic approach for evaluating the security of IoT messaging protocols[C]//Proc of the 30th USENIX Security Symposium. Berkeley, CA: USENIX Association, 2021:4205-4222.
- [10] Basin D, Cremers C, Dreier J, et al. The Tamarin manual[EB/OL]. <https://tamarin-prover.github.io/>.
- [11] Coppen R, Craggs I. ISO/IEC 20922:2016, OASIS message queuing telemetry transport (MQTT) TC[S/OL]. (2014-10-29). <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>.
- [12] 李峻辰, 程光, 杨刚芹. 基于网络流量的私有协议逆向技术综述[J]. *计算机研究与发展*, 2023, 60(1):167-190. (Li Junchen, Cheng Guang, Yang Gangqin. Private protocol reverse engineering based on network traffic: a survey[J]. *Journal of Computer Research and Development*, 2023, 60(1):167-190.)
- [13] 苗新亮, 常瑞, 潘少平, 等. 可信执行环境访问控制建模与安全性分析[J/OL]. *软件学报*. (2023-01-19). <https://doi.org/10.13328/j.cnki.jos.006612>. (Miao Xinliang, Chang Rui, Pan Shao-ping. Modeling and security analysis of access control in trusted execution environment[J/OL]. *Journal of Software*. (2023-01-19). <https://doi.org/10.13328/j.cnki.jos.006612>.)
- [14] Lowe G. A hierarchy of authentication specifications[C]//Proc of the 10th Computer Security Foundations Workshop. Piscataway, NJ: IEEE Press, 1997:31-43.
- [15] Lamacchia B, Lauter K, Mityagin A. Stronger security of authenticated key exchange[C]//Proc of International Conference on Provable Security. Berlin: Springer, 2007:1-16.
- [16] Dolev D, Yao A. On the security of public key protocols[J]. *IEEE Trans on Information Theory*, 1983, 29(2):198-208.
- [17] Arvind S, Narayanan V A. An overview of security in coap: attack and analysis[C]//Proc of the 5th International Conference on Advanced Computing & Communication Systems. Piscataway, NJ: IEEE Press, 2019:655-660.
- [18] Niruntasakrat A, Issariyapat C, Pongpaibool P, et al. Authorization mechanism for MQTT-based Internet of Things[C]//Proc of IEEE International Conference on Communications Workshops. Piscataway, NJ: IEEE Press, 2016:290-295.
- [19] Sahmi I, Abdellaoui A, Mazri T, et al. MQTT-PRESENT: approach to secure Internet of Things applications using MQTT protocol[J]. *International Journal of Electrical & Computer Engineering*, 2021, 11(5):4577-4586.
- [20] Lawrence T, Li Fagen, Ali I, et al. A computationally efficient HMAC-based authentication scheme for network coding[J]. *Telecommunication Systems*, 2022, 79(1):47-69.